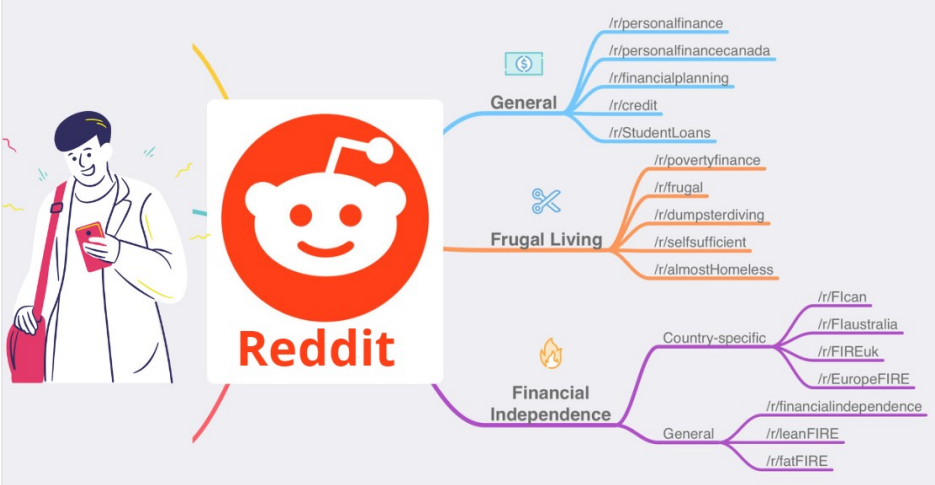
ML4N - Group Project 7

Patterns in comments from Reddit social network

Clarifications for this project can be asked to **Franco Galante**: franco.galante@polito.it



Reddit is an entertainment, social networking, and news website where registered community members can submit content, such as text posts or direct links, making it essentially an online bulletin board system. Registered users can then vote submissions up or down to organize the posts and determine their position on the site's pages. Content entries are organized by areas of interest called "subreddits". The subreddit topics include news, gaming, movies, music, books, fitness, food, and photo sharing, among many others.

When items (links or text posts) are submitted to a subreddit, users can vote for or against them (upvote/downvote). Each subreddit has a front page that shows newer submissions that have been rated highly. Users can also post comments about the submission and respond back and forth in a conversation-tree of comments; the comments themselves can also be upvoted and downvoted. The front page of the site itself shows a combination of the highest-rated posts out of all the subreddits a user is subscribed to.

The Reddit website has an API and its code is open source. In July 2015, a Reddit user identified as Stuck_In_the_Matrix made public a dataset of Reddit comments for research. Each data point contains comment text, score, author, subreddit, position in comment tree and other fields that are available through Reddit's API.

We selected a total of 20k users, with their 1.4 million comments. Among these, we selected 5000 with **self-reported gender** for the supervised task, and the remaining 15000 for the unsupervised one. You will be provided with 3 different csv data files:

- data_supervised.csv contains all comments of the users selected for supervised learning.
- target_supervised.csv contains the genders of the users selected for the supervised part. ‘1’ stands for female, ‘0’ for male.
- data_unsupervised.csv contains the comments of the users selected for the unsupervised task.

Each comment has the following **structure**:

- **author**: contains the username of the author
- **subreddit**: contains the subreddit in which the comment was posted
- **created_utc**: contains the date of submission in UNIX time format.
- **body**: contains the text of the comment.

Section 1 – Data exploration and pre-processing

Load the dataset, manipulating the data and answering the following questions:

1. **Data Exploration.**
    a. How many different attribute values do you observe in each feature? (e.g. how many subreddits are there?) Is there any missing or duplicated data?
    b. How does the empirical distribution of the number of characters in each comment look like? How is the distribution of the number of comments per author?

2. **Data Cleaning and Text Standardization.**
    a. Clean and uniform the text.
    b. Stop words are not contributing much to our ML tasks, such as "the", "a", since they carry very little information. Take care of these kinds of words.
    c. Reduce words to their base or root form using Stemming/Lemmatization. This helps in reducing inflected words to a common base form.

3. **Text Vectorization.**
    a. *Only for the supervised task (data_supervised.csv):* Group and join all comments of the same author, creating a "new" dataset to be used for the *supervised task* (Section 2).
    b. As ML algorithms struggle to handle directly the raw textual data. You are required to convert the text into numerical representations (**vectors**) through Bag of Words (BoW).
    c. Another way to assign a vector representation to each **user/comment** is TF-IDF (Term Frequency-Inverse Document Frequency).
    d. Can you observe and explain the differences between the numerical representations generated by BoW and TF-IDF?

4. **Reduce the complexity of your dataset.**
    a. **Threshold-based**: BoW and TF-IDF representations use the same large vocabulary, which can result in a very high-dimensional feature space. This may lead to issues like the *curse of dimensionality* or even *out-of-memory errors* when processing the data. Use the BoW representation to identify thresholds to reduce the dimensionality of the feature space of either the BoW itself or the TF-IDF representation.
    b. **Dimensionality Reduction**: Try applying PCA to reduce the dimensionality of your features. Does it work? How many components would you keep and why? Do the components you obtain with PCA have a physical interpretation?
    Note: The number of features is very large and both TF-IDF and BoW **matrix are sparse**.

Section 2 – Supervised learning – Classification

One of the user attributes that is not natively supported by the Reddit platform is the gender. However, in some subreddits, users can **self-report their genders** as part of the subreddit rules. Users that self-reported their gender are selected from the dataset. Your goal is to **predict the gender** of these users. **Note!** Do not use column **author** as a feature and use the dataset you have just obtained for each of the Reddit users (the BoW and the TF-IDF reduced representations).

1. **Prepare the dataset for the training task.**

2. **ML Model Training and Evaluation.**
    a. Choose at least 3 ML methods. Perform model training with the default parameter configuration, using both the BoW and the TF-IDF representations, evaluate the performance on both training and test set. Output the confusion matrix and classification report. Which model and which representation generates the best performance?
    b. Do you observe **overfitting or under-fitting**? If so, why does this happen and how to deal with this issue? Provide evidence for your claims.
    c. Tune the hyper-parameters of the models through cross-validation. How does performance vary? What is your choice of the best model and hyper-parameters according to *training cost* (you can use training time as a "cost" proxy) and performance? Select your "best" pipeline.
    d. Group the test users according to their number of comments (e.g., 1–5, 6–20, 21–100, >100) and report the performance of your best pipeline for each group. How many comments do you need to reliably infer the gender of a user?

3. **Shortcut Analysis.**
    a. Some subreddits are strongly associated with one gender (e.g., AskMen, AskWomen, MakeupAddiction), so the subreddits in which a user posts might be enough to infer the gender. For each user, create a feature vector where each column represents the **fraction of times the user has posted in the i-th subreddit**. Train your best model using only these features: how does it perform compared with the text features? If you include both text and subreddit features in the training, does your model improve?
    b. Using only the training users, identify the most gender-skewed subreddits (**motivate your selection criterion**). Remove the comments posted in these subreddits, as well as the comments in which users explicitly reveal their gender (e.g., "my husband", "as a guy")[1], and repeat the training and evaluation. How much does the performance drop in each case? What is your model actually learning?

Section 3 – Unsupervised learning – Clustering

In this unsupervised learning task, the objective is to cluster Reddit comments based on the **words used** within them. The aim is to **identify patterns** and group comments with similar words together. Given the size of the dataset, you can work on a sub-sampled version of the dataset (motivate your sampling).

Note: Use only the column **body** (i.e. the textual data) to extract the features used in this section, do not feed "author", "subreddit" and "created_utc" to the model.

To extract features, adapt your *best preprocessing pipeline* developed in Section 1 at a per-comment level. Keep in mind the problems mentioned in Section 1.4 regarding features dimensionality reduction. Choose at least 2 Clustering Algorithms, and for each of them:

1. **Determine the number of clusters.**
   This can be done using methods like the elbow method or silhouette analysis. Explain your reasoning.

2. **User Distribution Analysis.**

[1] Report either the list you used or the methodology for the selction.

a) Analyze how the users are distributed (each user can be assigned to more clusters because we are clustering comments). Show the Empirical Cumulative Distribution Function (ECDF) of number of clusters assigned to each user.
b) Use the first two components of the PCA (or SVD) of the features used for clustering. Then visualize on these two dimensions to gain insights into their distribution and separability.

3. **Cluster analysis.**
   a) Analyze the characteristics of each **cluster**. This might involve examining the most frequent words in each cluster (try word cloud). **Hint**: the most used words are not always the most representative, for example word *'the'* will be very popular in every cluster if you didn't remove it from the dataset. Try to understand which are the most representative.
   b) **Do clusters reflect subreddit division**? Finally, analyze and visualize how the subreddit are distributed among the determined clusters. Are there pure clusters where only few **related** subreddits end up?

4. **Language Model Embeddings**
   Use **Language Models** to get text **embeddings** from the comments. You can use either Bert or Doc2Vec, it is sufficient to select one of the two:
   a) **If you choose Doc2Vec**, train Doc2Vec based on the comments. Save the embedding vectors or the model itself for the next steps.
   b) **If you choose Bert**, take the pretrained Bert model like in this example from HuggingFace. You only need the embedding vectors from the model rather than other down-stream task capabilities, so just take the **base** model and extract the embeddings for the following steps. And you are not required to do fine-tuning here. You can use: the Bert, DistillBert (light version of Bert, smaller, faster), or any other Bert variations.
   c) Perform the clustering again with these features, do you think your clustering improved?

Section 4 – Sentiment Analysis

Reddit comments often carry emotional tone. In this exercise, you will explore these aspects to enrich your understanding of text analytics.

1- **Sentiment Analysis (classical):** Use a pre-trained sentiment analysis model (e.g., VADER from NLTK or HuggingFace transformers) to classify each comment in the *supervised* dataset as **positive**, **negative**, or **neutral**.
   a. Note that lexicon-based models such as VADER work on the raw text: removing stopwords (e.g., "not", "never", "no"), stemming or removing punctuation may be harmful. Adapt your preprocessing pipeline accordingly and describe it carefully.
   b. What is the overall sentiment distribution across all comments?
   c. Visualize sentiment distribution per subreddit using bar charts or heatmaps.
   d. Does sentiment correlate with gender? For example, do male or female users tend to post more positive or negative comments?

2- **Sentiment Analysis - LLM – zero-shot:** Try zero-shot classification using a small LLM, such as qwen3:1.7b or gemma2:2b (you can use larger models, e.g., llama3.2 (3B) or qwen2.5:3b, if your system allows it). You can use the Ollama Python library to download and use these open-source models.

a. Define candidate labels: ["positive", "negative", "neutral"] and develop a prompt to let the LLM classify comments and provide you with a single label.
b. Use your prompt to classify the comments with the LLM. Use only a **small subset** of the dataset (running an LLM may be resource-intensive!).
c. For the reduced dataset, manually inspect 80 comments and label them ("positive", "negative", "neutral"). Choose if taking the comments from different subreddit or the same one, motivate your reasoning. **Compare** the zero-shot predictions with the sentiment analysis model predictions and your labels. According to this small-scale experiment which methodology seems the most promising?

Project idea acknowledgment: Giordano Paoletti: giordano.paoletti@polito.it